

M7: struct、class、成员、构造与复制

CSC3002 基础讲义 · 第 3 册 · 前置：M2 的函数与引用、M3 的对象与指针

核心问题：定义一种新类型以后，每个对象各自拥有什么，创建对象和修改已有对象又有什么区别？学完应能读懂成员访问、构造函数、this、作用域限定、复制构造与赋值，并解释 A1 的 str 为什么是一个独立类型。

主线：相关数据需要放在一起 → 定义类型 → 创建对象 → 成员与访问控制 → 构造与初始化 → this → const 成员函数 → 类外定义 → 复制和赋值 → A1 接口。采用 C++17。先完整建立简单对象模型，最后以 E22 Q30 补一个有明确边界的继承与虚函数入门连接，不把它当成全部后续 OOP 内容。

阅读依据与课程定位

这是对实际课程材料重新核对后的修订版，核对日期为 2026-10-08。L 后的数字是课件代号；“第几张”按文件实际幻灯片顺序计数，从封面开始，不沿用旧模板页脚。E22 页码为 PDF 页码。

- L2: 2-CSC3002_Lecture Notes Functions and Libraries Part 1(2).pptx
- L4: 4-CSC3002_Lecture Notes_Strings.pptx
- L6: 6-CSC3002_Pointers and Arrays(4).pptx
- L7: 7-CSC3002 Dynamic memory management.pptx
- E22: CSC3002 Final 2022 Spring.pdf
- A1: assignment1.pdf

本册学习任务	原材料定位	要能独立完成的动作
class、object、字段与方法	L4 第 19-20 张；L2 第 37 张	区分类名、对象与每个对象自己的成员
成员访问、this、类外定义	L6 第 30-33 张	读懂点号、箭头与作用域限定符
const 成员与初始化	L7 第 22-23 张	区分成员初值、构造初始化、函数体赋值
复制、赋值与资源关系	A1 第 1 页；E22 Q29；L7 第 10-17 张	先问新对象是否正在创建，再判断复制哪一层
OOP 术语与访问说明符	E22 Q21、Q24；Q30 为入门连接	解释术语并分步手推虚函数例题

课件决定本册的概念范围；旧卷用于练题型和深度，不预测今年题目或分值。本册的解释、状态表和答案由教学推导得到；未找到官方答案册，因此不称为官方标准答案。保留的最小例子与一般练习属于自编教学补充；涉及原材料时逐项标明“课件例题”“原题代码”或“题意转述”。

当前 L1 第 2 张写的是闭卷，E22 第 1 页写的是可带一张双面 A4 的限制性开卷，两者不能混作今年规则。六册只覆盖 C++ 基础这一段，旧卷的数据结构、算法和部分继承多态内容仍需后续学习。

1. 为什么需要自己定义类型？

要表示二维点，两个无关的整数 x、y 很容易失去所属关系：哪个 x 对应哪个 y？**自定义类型允许把属于同一个概念的数据，以及相关操作，组合为一个类型。**

```
struct Point {
    int x = 0;
    int y = 0;
};
```

`struct` 开始定义一个类类型；`Point` 是类型名；花括号内声明成员；末尾分号结束类型定义。`x`、`y` 是每个 `Point` 对象中的非静态数据成员，成员初始值为 0。

这段代码定义“`Point` 对象的形式”，尚未单独创建名为 `p` 的 `Point` 对象。把类型当作设计说明可以帮助入门，但类型不是运行时某一个具体对象。

2. 类型和对象：Point 与 p 的差别

```
Point p;
Point q;
```

`p` 和 `q` 是两个 `Point` 对象，各自具有 `x`、`y` 成员。因为上述类型定义中有成员初始值，刚创建后两个对象的 `x`、`y` 都为 0。

```
p 对象：自己的 x，自己的 y
q 对象：自己的 x，自己的 y
```

`Point` 回答“是什么类型”；`p` 回答“这一次访问哪个对象”。同一种类型可以创建很多对象，正如 `int` 可以创建很多整数。**自定义类型并不让同名成员自动共享。**

```
p.x = 3;
q.x = 8;
```

这里分别修改 `p` 中和 `q` 中的 `x`，互不覆盖。不能笼统说“把 `Point` 的 `x` 改成 3”，因为需要指出是哪一个 `Point` 对象。

3. 成员是什么？点号怎样读？

成员是声明在类定义中的组成部分；这里的数据成员保存对象的数据，成员函数提供相关操作。

```
Point p;
p.x = 3;
p.y = 4;
int sum = p.x + p.y;
```

`p.x` 的点号表示“访问对象 `p` 的成员 `x`”。最后一行读取两个整数成员并相加，`sum` 为 7。访问权限满足时，`p.x` 也能放在赋值左边，因为它表示一个整数成员对象。

点号左边先确定对象，右边再确定该对象的哪个成员。这样就把 M3 的“访问哪个对象”扩展到了“访问对象里的哪个成员”。

4. 指针访问成员为什么使用 ->?

```
Point p;
Point* ptr = &p;
ptr->x = 9;
```

`Point*` 是指向 `Point` 的指针类型；`ptr` 保存 `p` 的地址；`ptr->x` 通过指针访问目标对象的 `x`。对于这里的内建指针和成员访问，它相当于 `(*ptr).x`。

括号不能省成 `*ptr.x`：点号会先与 `ptr` 结合，尝试访问指针对象的成员 `x`，结构就错了。箭头把“先解引用，再访问成员”的常见操作写得更直接。

使用箭头同样要求 `ptr` 有效、目标仍存活。它不会自动让空指针变成一个新 `Point`。

5. struct 和 class 是两套对象机制吗？

在本册无继承例子中，它们都定义类类型，也都能有数据成员、成员函数和构造函数。**最直接的区别是默认成员访问权限：struct 默认 public，class 默认 private。** 默认继承访问也不同，但本册不展开继承。

```
struct A {
    int value = 0;    // 默认 public
};
class B {
    int value = 0;    // 默认 private
};
```

`public` 成员可以在满足其他语言条件时由类外代码访问；`private` 成员只能由有权限的代码访问，例如本类成员函数。`A a; a.value = 3;` 可以；直接 `B b; b.value = 3;` 不可以。

不是“struct 只能放数据，class 才能有函数”，也不是“一写 class 就必须有继承”。先掌握同一套对象模型，再考虑设计风格。

6. private 为什么有用？先给一个实际需求

希望计数器的修改都经过 `add` 操作，便可隐藏其内部 `value`，让调用者只通过公开函数使用它。

```
class Counter {
private:
    int value = 0;
public:
    void add(int amount) {
        value += amount;
    }
    int get() const {
        return value;
    }
};
```

`private:` 和 `public:` 是访问区段标签，对随后成员生效。调用者写 `c.add(3);`，类内部决定如何更新 `value`；调用者通过 `c.get()` 读取结果。当前简单计数器不限制 `amount` 的正负，不能凭空声称它保证非负。

封装在这里的实际作用是把访问入口与内部表示分开，而不是为了背一段面向对象定义。

7. 成员函数是什么？调用时操作谁？

非静态成员函数是针对某个对象调用的函数。 它除了显式参数，还与发起调用的对象关联。

```
Counter a;
Counter b;
a.add(3);
b.add(7);
```

执行 `a.add(3)` 时，函数中的 `value` 指 `a` 的成员；执行 `b.add(7)` 时，同样的函数代码中的 `value` 指 `b` 的成员。最后 `a.get()` 为 3，`b.get()` 为 7。

用 M2 语言解释：`amount` 是这次调用的显式参数，3 或 7 是实参；当前对象则决定没有限定名的 `value` 属于谁。接下来的 `this` 就是表达这个当前对象的方式。

8. 构造函数是什么，为什么不是普通 setter?

构造函数用于初始化正在创建的类对象。 它与类同名，不声明返回类型；对象初始化时选择适当构造函数。

```
struct Point {
    int x;
    int y;
    Point(int xx, int yy) : x(xx), y(yy) {}
};
Point p(3, 4);
```

这是新的一份独立示例，不与前面的 `Point` 重复放在同一作用域。`Point(int xx, int yy)` 声明构造参数；冒号之后是成员初始化列表；`x(xx)` 用 `xx` 初始化成员 `x`，`y(yy)` 同理；最后 `{}` 是空函数体。

创建 `p` 时，3 初始化 `xx`，4 初始化 `yy`，再由它们初始化 `p.x` 与 `p.y`。此后 `p` 已经有一组可用初始状态。

`setter` 操作一个已经存在的对象；构造函数参与新对象的初始化。不能在构造函数前写 `void`，因为它不是“没有结果的普通同名函数”。规则见 [构造函数](#)。

9. 初始化列表和函数体赋值有什么区别?

```
struct Box {
    int value;
    Box(int n) : value(n) {}
};
```

这里 `value` 从一开始就按 `n` 初始化。若改成 `Box(int n) { value = n; }`，函数体中的语句是在成员初始化阶段之后执行赋值。对这个简单 `int`，最终读到的值可以相同，但步骤不同。

区别会在 `const` 成员、引用成员等情况下直接影响合法性：它们需要在初始化阶段获得合适初值，不能等到函数体里再用普通赋值“补初始化”。

多个成员按**声明顺序**初始化，不按初始化列表书写顺序。因此让初始化列表按成员声明顺序写，是为了使代码与实际顺序一致。当前例子不引入相互依赖的复杂成员。

10. 默认构造函数到底“默认”什么?

可不提供实参就调用的构造函数称为默认构造函数。 最常见形式是无参数构造函数，也可以是所有参数都有默认实参的构造函数。

```
struct Box {
    int value = 0;
    Box() = default;
    Box(int n) : value(n) {}
};

Box a;
Box b(8);
```

`Box() = default;` 请求使用编译器默认生成的构造行为；这里还存在成员初始值，因此 `a.value` 为 0。b 的构造函数显式初始化 `value` 为 8。

若一个类只有 `Box(int n)` 且没有默认实参，又没有声明默认构造函数，就不能凭空假定 `Box a;` 一定可用。`= default` 也不普遍等于“所有数据无条件清零”；本例的 0 来自成员初始化写法。

11. this 是什么？它怎样接到指针？

在这里的非静态成员函数中，**this** 是指向当前对象的指针表达式。对非 `const Box` 成员函数，可把它读成指向当前 `Box` 的 `Box*`。

```
struct Box {
    int value = 0;
    void set(int value) {
        this->value = value;
    }
};
```

参数也叫 `value`，会遮住不加限定的成员名字。`this->value` 明确访问当前对象的成员；等号右侧的 `value` 是参数。因此 `b.set(9)` 把 `b.value` 改为 9。

若错误写成 `value = value;`，两侧都指参数，并没有修改成员。若参数改名为 `n`，就可写 `value = n;`。`this` 的用途是明确当前对象，不必每次成员访问都强制写它。可核对 [this 表达式](#)。

12. *this 又是什么？为什么可以 return *this?

既然 `this` 指向当前对象，`*this` 就表示当前对象本身。它仍是普通的指针与解引用模型。

```
struct Box {
    int value = 0;
    Box& set(int n) {
        value = n;
        return *this;
    }
};
```

返回类型 `Box&` 表示返回一个引用；`return *this;` 让这个引用表示调用中的原对象，不创建新的 `Box` 副本。

```
Box b;
Box& r = b.set(5);
r.value = 8;
```

`b.set(5)` 先把 `b.value` 改为 5，再提供对 `b` 的引用；`r` 绑定到 `b`，因此之后 `b.value` 为 8。只要 `b` 仍存活，返回对调用者这个对象的引用就可继续使用；这与返回函数内即将消失的局部对象引用不同。

13. const 成员函数末尾的 const 限制谁？

```
int get() const {
    return value;
}
```

参数列表后的 const 表示这是 const 成员函数，通过当前对象访问普通非 mutable 数据成员时不能修改它们。它允许对 const 对象调用这一只读操作。

```
const Counter c;
int n = c.get();
// c.add(1); // 错误示例：add 不是 const 成员函数
```

这里的 const 不是“返回一个 const int”，位置不同，含义不同。也不要把 const 成员函数理解成绝对没有任何外部副作用；例如输出日志仍可能发生。本例只需要它表示不通过当前对象修改普通 value 成员。

14. :: 是什么？类外定义为什么要写它？

作用域限定符 :: 指明一个名字属于哪个作用域。 点号在具体对象上找成员，:: 在命名空间或类作用域里定位名字，不能混用。

```
struct Box {
    int value = 0;
    void set(int n);
};

void Box::set(int n) {
    value = n;
}
```

类内 void set(int n); 是成员函数声明；类外的 Box::set 告诉编译器，这个定义属于 Box 的 set，而不是新定义一个无关的全局 set。

Box:: 本身没有指定某一个 b 对象；具体当前对象仍由 b.set(3) 这样的调用决定。后面 M8 的 std::cout、mystring::str 也会使用同一限定符定位名字。

15. copy 是什么？先区别“复制内容”和“成为别名”

```
struct Pair {
    int left = 0;
    int right = 0;
};
Pair a;
a.left = 3;
a.right = 4;
Pair b = a;
b.left = 9;
```

默认复制使 b 的两个整数成员取得 a 对应成员的值。a、b 仍然是两个对象，最后 a.left 为 3，b.left 为 9。若写 Pair& r = a;，r 才是原对象的引用，不是副本。

这里能够独立修改整数成员，是成员本身按值保存数据。若成员是裸指针，默认复制通常只复制指针值，不会自动把指向的数据也复制一份；M9 会讲为什么拥有动态资源的类需要考虑这个区别。

16. 复制构造和复制赋值：先问对象是否已经存在

复制构造从另一个对象初始化一个新对象；复制赋值改变一个已经存在的对象。 外观都可能带等号，但语境决定它们不是同一操作。

```
Pair a;
a.left = 3;
Pair b = a;      // 新对象 b: 复制初始化
Pair c(a);       // 新对象 c: 直接初始化，选择复制构造
Pair d;
d = a;           // 已有对象 d: 复制赋值
```

`Pair b = a;` 是声明中的初始化，不能只因为有 `=` 就认定调用赋值运算符。对本例，编译器提供成员复制行为；类也可以自行声明复制构造和赋值函数。

常见复制构造声明为 `Box(const Box& other);`：函数名同类名、不写返回类型，参数借用来源对象。复制赋值常见声明为 `Box& operator=(const Box& other);`：修改已有当前对象，通常返回它的引用。

标准对复制构造有更完整的规则；本册先把创建与修改的时刻分清。见 [复制构造](#)。

17. 为什么赋值通常返回原对象引用？

```
struct Box {
    int value = 0;
    Box& operator=(const Box& other) {
        value = other.value;
        return *this;
    }
};
```

这是教学用最小类，只保存一个 `int`。`operator=` 是赋值操作的函数名；`const Box& other` 让函数读取来源；`value = other.value` 改当前对象；`return *this` 返回当前对象引用。

对 `a = b`，当前对象是 `a`，`other` 引用 `b`。返回 `Box&` 允许调用结果继续表示 `a`，例如 `Box& r = (a = b);`。为什么链式赋值可行，会在 M8 把运算符模型完整拼起来。

对这个单 `int` 复制，自赋值 `a = a` 没有问题；不必为了讲解先加一套未经需要的保护代码。资源类的复制约束要根据其实际所有权再判断。

18. 数组作为成员，和裸指针作为成员不同

```
struct Text {
    char data[4] = "abc";
};
Text a;
Text b = a;
b.data[0] = 'X';
```

当这里使用编译器生成的复制操作时，数组成员的元素被复制到新对象自己的数组中。`a.data` 的内容仍为 `abc`，`b.data` 为 `Xbc`。

这不与“内建数组不能直接整体赋值”矛盾：`b.data = a.data`；不是合法内建数组赋值，而类的默认复制行为可以逐元素复制其数组成员。

如果把成员改为 `char* data`，默认复制保存的是同一个指针值，对象关系就不同。先识别实际成员类型，不能只凭它们都叫 `data` 判断。

19. 当前 A1 的构造写法怎样读？

当前 `assignment1/string.h` 中 `str` 的相关成员是：

```
str& operator=(const str& src);
str& operator=(const char* src);
str(const str& other) { *this = other; }
str(const char* src) { *this = src; }
str() = default;
```

这些摘录都属于 `struct str` 内部。两个 `operator=` 是赋值函数声明，参数分别为 `str` 来源和 C 字符串来源。两个构造函数在自己的函数体里复用相应赋值操作。

`str(const str& other) { *this = other; }` 的外层仍然是复制构造：正在创建新 `str`。执行到函数体时，当前对象的成员初始化阶段已经完成，再对 `*this` 调用赋值操作复制内容。**构造函数内部使用赋值，不会让外层创建动作变成普通赋值。**

`str() = default`；使用默认构造行为，配合 `char data[maxlen + 1] = {'\0'}`；的成员初始值，使默认对象具有零初始化的字符存储。这些结论依据当前真实头文件，不代替其他版本接口。

20. 完整程序：成员归属、复制和 this

```
#include <iostream>

struct Box {
    int value;
    Box(int n = 0) : value(n) {}
    Box& set(int n) {
        this->value = n;
        return *this;
    }
    int get() const { return value; }
};

int main() {
    Box a(3);
    Box b = a;
    Box& r = a.set(8);
    r.set(9);
    std::cout << a.get() << ' ' << b.get() << '\n';
}
```

a 初始 value 为 3；b 是独立复制，value 为 3；`a.set(8)` 返回对 a 的引用；`r.set(9)` 操作的仍是 a。输出 9 3。

回到 lecture notes: 用课件的对象把模型接起来

A. L4 第 20 张的 Point: 表示和操作在同一个类型里

课件的 Point 类包括公开 x、y, 以及 length 和 operator+ 等成员。它把“保存两个坐标”与“对坐标做操作”结合起来。对应本册前面的最小 Point, 可以分三层读: 类定义说明形式; Point pt 创建一个对象; pt.x 访问该对象里的字段。

L6 第 30-31 张进一步写 Point* pp=&pt。pt.getX()、(*pp).getX()、pp->getX() 在存在对应公开成员函数时可以访问同一个对象。pp 本身是指针, 不是 Point; 不能写 pp.getX()。先确定左侧对象还是指针, 再选择点号或箭头。

L6 第 33 张的构造函数用 this->x=x、this->y=y 解决参数与成员同名。this 是当前对象指针, 因此等号左侧是对象成员, 右侧是参数; 它不是“当前类的全部对象”。

B. L7 第 23 张: 初始值可以被构造初始化替代

下面按课件机制整理, 省略未用到的另一个成员:

```
class A {
public:
    const int c1 = 1;
    A(int xx) : c1(xx), x(xx) {}
    int getC1() const { return c1; }
    int getX() const { return x; }
    void add(const int xc) { x += xc; }
private:
    int x;
};
```

执行 A a(5), c1 的值从创建开始就是 5, 因为构造函数初始化列表为它提供了初值。不是先把一个已存在的 const 成员设成 1, 再偷偷改成 5; 成员默认初始值只在没有对应显式成员初始化时使用。

再调用 a.add(100), x 从 5 变成 105, c1 仍为 5。参数 xc 的 const 限制函数体给参数副本重新赋值, 不会让整个 a 只读。getX 后面的 const 则限制通过当前对象修改普通成员。**同一个关键字位置不同, 约束对象不同。**

C. 抽象、封装、继承、多态先讲它们解决什么问题

E22 Q21 (第 5 页) 列出四个常见 OOP 术语。先按最小含义理解:

术语	它是什么、有什么用	本册可见的连接
abstraction 抽象	用一组操作表达概念, 让使用者不必逐步处理内部细节	string 的 length、Point 的方法
encapsulation 封装	把状态与操作组织在类型里, 并控制访问入口	private x 与公开 getX/add
inheritance 继承	新类包含基类部分并建立类型关系	后面的 B:public A
polymorphism 多态	同一接口可表现出不同具体行为	虚函数通过基类指针选实际覆盖实现

按课程中常见的“四项 OOP 特征”分类, Q21 预期四项均选; 这不是说只有 OOP 能有抽象或封装, 其他范式也使用这些思想。答题时能用一个对象例子解释术语, 比只背四个英文词更稳。

D. 复制值、复制资源、切片是三件需要分开的事

默认复制一个含 `int` 成员的对象，会复制整数值；默认复制含 `char data[4]` 的对象，会复制数组成员的元素；默认复制含 `int*` 的对象，只复制指针值。这些都是成员类型决定的行为，不存在“所有默认复制必然只复制地址”的规则。

shallow copy (浅复制) 在本课程资源问题中指复制了指针等表层信息，两份对象仍关联同一资源。**deep copy (深复制)** 则为目标建立独立资源并复制其内容。若目标负责 `delete`，共享裸地址可能带来重复释放，M9 再完整展开。

slicing (对象切片) 是把派生对象按值放进基类对象时，只保留基类部分。它与两对象共享一个动态数组不同；只重载普通复制构造和 `operator=`，不能普遍让基类对象容纳任意派生部分。

练习

题 1：对象和成员

`struct P { int x = 0; }; P a; P b; a.x = 5;`。P、a、a.x 分别是什么？b.x 是多少？

题 2：构造还是赋值？

假设 Box 有合适的默认构造、复制构造和复制赋值。分别判断：`Box b = a;`、`Box c(a);`、`Box d; d = a;`。

题 3：this 解决什么歧义？

```
struct Box {
    int value = 1;
    void bad(int value) { value = value; }
    void good(int value) { this->value = value; }
};
```

对同一个对象先 `bad(7)`，再 `good(8)`，成员 `value` 依次是多少？

题 4：返回引用

沿用第 20 节 Box，执行 `Box a(2); Box& r = a.set(6); r.set(10);` 后，r 和 a 的关系是什么？a 的值是什么？

题 5：指针成员的复制

```
struct View { int* p; };
int n = 4;
View a{&n};
View b = a;
*b.p = 9;
```

n 是多少？a.p 和 b.p 指向谁？有没有自动复制一个新整数？

题 6：读 A1 的复制构造

`str(const str& other) { *this = other; }` 中 other、this、*this 分别表示什么？执行 `str b(a);` 是创建对象还是修改已有 b？

练习详细解析

题 1 解析

P 是类型；a 是该类型的一个对象；a.x 是 a 的整数成员对象。a.x 的赋值只修改 a 的成员，b.x 仍为 0。若误答 b.x = 5，是把非静态成员当成所有实例共享的数据。

题 2 解析

前两种都在创建新对象，使用复制构造来初始化 b 或 c；第三种先默认构造 d，再用赋值修改已有 d。判断依据是是否处在声明初始化新对象的语境，不是等号是否出现。

题 3 解析

bad 内两处 value 都解析为参数，因此成员保持 1。good 左边 `this->value` 明确指向当前对象成员，右边 value 为参数，所以成员变成 8。修复也可通过把参数改名为 n，使角色不重名。

题 4 解析

set 返回 `*this` 的引用，r 因此绑定到 a。r.set(10) 操作同一对象，a.get() 为 10。没有新建一个独立的 r 所有 Box 对象；引用名只是访问已有对象。

题 5 解析

默认复制把 a.p 的指针值复制到 b.p，两者都指向 n。`*b.p = 9` 改 n，所以 n = 9，a 也能经 a.p 读到 9。a、b 是不同 View 对象，但它们指向的目标相同。不同对象不保证它们关联的资源完全独立。

题 6 解析

other 是对来源 str 的只读引用；this 指向正在构造的目标 str；`*this` 表示该目标对象。`str b(a)`；创建并初始化 b，外层是复制构造。构造函数体内部的赋值表达式负责复用内容复制操作，与外层创建动作分层理解。

下一册从哪里继续？

现在已经能把自定义类型当成普通类型阅读：有对象、有成员、有参数和返回类型。M8 只需在这个模型上解释 `operator+`、`operator+=`、输出流和头文件组织，不需要先进入复杂继承体系。

旧卷与课件练习详细解析

O1. E22 Q24: 哪个是成员访问说明符?

原题 (第 6 页) 选项是 public、protected、friend、private。选 **a、b、d**。

public、protected、private 用于指定相应访问控制。protected 允许基类和派生类中满足规则的成员访问，并对普通类外代码隐藏；入门时先知道它与继承有关。friend 是友元声明，给指定函数或类访问权限，本身不是一个“friend 访问区段”。不能因为某个词出现在类里就认为它是访问说明符。

struct 和 class 的默认访问不同，但它们都可以显式写这些访问区段。由此也能纠正课件“class 比 struct 更有能力”的历史性简述：在 C++ 中，两者都能有成员函数、构造函数等，不能拿 C 的结构体限制去约束 C++ struct。

O2. E22 Q29: 复制构造、赋值与题面中的 slicing

原题 (第 7 页) 问为避免默认浅复制或 slicing，应重载哪些操作，选项依次为复制构造、赋值 =、插入 <<、选择 []。

按课程针对复制控制的意图，预期选 a、b。 新对象复制从复制构造进入，已有对象内容复制从复制赋值进入；<< 管输出，[] 管选择，都不能替代这两个入口。

但题面把 shallow copy 与 slicing 合在一句话里不够精确。为拥有动态资源的类实现适当的复制构造和赋值，可以实现独立内容复制；基类按值接收派生对象导致的切片，则通常需要避免按值切成基类、改用合适的引用/指针或多态复制设计。不能把“写两个函数就普遍消除切片”当定理。A1 的数组成员已经内嵌在各自对象里，分析时还必须看实际接口。

O3. E22 Q30: 先补两个前置概念，再推原题

这是后续继承、多态模块的入门连接。**基类**提供被继承的部分；**派生类**包含该基类部分并可增加自己的成员。构造派生对象时，先构造其基类部分。virtual 指定虚函数；通过合适的基类指针/引用调用时，可以选择实际对象中的覆盖实现。

原题代码 (第 7 页)：

```

#include <iostream>
using namespace std;
class A {
public:
    int a;
    A() { a = 1; }
    virtual void display() { cout << a << endl; }
};
class B: public A {
public:
    int b;
    B() { b = 2; }
    void display() { cout << a << b << endl; }
};
class C: public B {
public:
    int c;
    C() { a = 2; c = 3; }
    void display() { cout << a << b << c << endl; }
};
int main() {
    C oC;
    B* pb = &oC;
    pb->display();
    return 0;
}

```

`class B: public A` 中冒号开始继承列表，`public` 指此处的继承访问；不是构造函数的成员初始化列表。`B* pb=&oC` 保存指向该对象中 B 基类部分的指针，并没有创建一个按值复制的 B 对象，所以这里不发生前面说的切片。

阶段	a	b	c	发生了什么
A 构造完成	1	尚未设值	尚未设值	先建立 A 部分
B 构造完成	1	2	尚未设值	B 设置自己的 b
C 构造完成	2	2	3	C 把继承的 a 改为 2，并设置 c

调用 `pb->display()` 时，A 中的 `virtual` 使这一同签名函数在继承链上保持虚函数性质，B/C 中不再次写 `virtual` 也会覆盖它。实际对象是 C，因此选择 `C::display`。连续输出 a、b、c，没有空格，显示：

```
223
```

选项 a=1、b=2、c=3、d=编译错误，题目问哪些数会出现在输出中，因此选 **b、c**。错误推法是只凭 `pb` 的声明类型就调用 `B::display`；另一种错误是忘记 C 构造里 a 已由 1 改成 2。

O4. 课件 `const` 例子的最小变式

沿用 A 类，执行 `A a(5); a.add(100);` 后，`getX` 为 105，`getC1` 为 5。如果写 `const A b(5)`，`b.getX()` 合法，而 `b.add(100)` 不合法，因为 `add` 不是 `const` 成员函数。这个结论来自访问路径与成员函数限定，不来自“带 `const` 的任何代码都不能运行”。